

CLINICAL AI CONTROL PLANE

Architecture Decision Records

CAG-03 · Six Accepted Decisions

Prepared by Guillermo (Bill) Cuevas · 2026-08-21

PHI PROTECTION · FULL AUDIT TRAIL · HIPAA ALIGNED

Clinical AI Control Plane: Architecture Decision Records

Six accepted decisions · each with context, decision, and consequences

SECTION 01

What the control plane builds

A governance gateway between a fleet of clinical AI assistants and the model layer: PHI redaction, per-tenant usage metering, policy-driven human review, provenance, and a Kafka-backed audit trail, exposed as a versioned REST API (`/api/v1`) secured as an OAuth2 resource server.

Decided and closed: governance is centralized in one plane rather than re-implemented per assistant. Per-assistant controls were rejected: controls drift apart, every new assistant repeats the work, and a compliance reviewer has no single place to verify behavior.

ADR 0001 · ACCEPTED

Centralized AI control plane

Context. The product direction is a fleet of assistants spanning billing, clinical, front office, patient, and support workflows. Each needs the same cross-cutting controls: PHI protection, per-tenant cost and abuse limits, human review for actions affecting a patient record, provenance, and a complete audit trail.

Decision. Assistants do not call the model directly. They call the control plane, which enforces redaction, metering, the review policy, provenance capture, and audit publishing, then delegates the model call through a provider seam. The plane owns the governed lifecycle of a suggestion from creation to a terminal decision.

Consequences. Governance is defined once and applied uniformly; a new assistant inherits the controls for free and a reviewer has a single system of record. The plane becomes a shared dependency on the request path, so it stays stateless at the web tier, backs state in Postgres, and keeps the model call behind an interface so it can be made asynchronous if the latency budget requires it.

ADR 0002 · ACCEPTED

Policy-driven human review

Context. Suggestions range from low-impact drafts to clinical and financial actions. Holding every output for approval slows low-impact paths without adding safety; reviewing nothing is unsafe for high-impact paths. Oversight should match the risk of the action.

Decision. Review is a policy decision, not a hard gate. The plane evaluates `app.governance.review-required-assistants` at creation time. Empty by default, so the default posture is autonomy; a risk-based list such as `CLINICAL,BILLING` turns on review for exactly those flows. The state machine and audit trail apply on every path, including auto-approval.

Consequences. No friction where it adds no value; human oversight exactly where risk justifies it, without code change. Every decision records an actor. The tradeoff: the policy itself becomes a governed artifact that must be reviewed and tested. It currently keys on assistant type and can extend to action risk, model confidence, or tenant configuration behind the same interface.

ADR 0003 · ACCEPTED

Kafka for the audit trail

Context. Every state change must produce a durable audit record for frameworks such as HIPAA and SOC 2. The trail must outlive the request, be queryable, and serve multiple consumers over time (compliance store, analytics, alerting). Direct Postgres writes in the same transaction and RabbitMQ were considered.

Decision. State changes publish an audit event to a Kafka topic keyed by suggestion id, so one suggestion's history stays ordered within a partition. A dedicated consumer persists events into a Postgres audit read model. Kafka retains the event log and lets independent consumer groups read the same stream without coupling.

Consequences. The trail is decoupled from the request transaction and fans out to new consumers without producer changes. The cost is an eventual-consistency window and a dual write between the database and Kafka. The production answer is a **transactional outbox**: the event is written to an outbox table in the same transaction and relayed to Kafka. The current code publishes inline behind a single component, so the move to an outbox is a contained change.

ADR 0004 · ACCEPTED

PHI redaction at the gateway boundary

Context. Assistant inputs can contain protected health information. Model providers may run outside the trust boundary, and even a provider under a business associate agreement should receive the minimum necessary data. A consistent removal point is needed rather than trusting each assistant or provider contract.

Decision. The plane redacts direct identifiers from inbound text at the gateway boundary, before the provider is called, for every assistant, through a single service behind a stable interface. The current implementation is pattern-based and illustrative; production replaces the implementation with a dedicated clinical detection service behind the same interface, without changing call sites.

Consequences. Redaction is applied once, uniformly, and independently of any provider contract, giving one place to test and improve detection. Tradeoffs: over-redaction can reduce model quality, and pattern-based detection is not sufficient alone for free-text clinical notes. Relying on a provider agreement alone was rejected: a contract does not reduce the data sent.

ADR 0005 · ACCEPTED

Vendor-neutral model provider seam

Context. The backend will change over time and must not be hardcoded into governance logic. The platform must not couple to a single vendor, needs a deterministic backend for local development and tests, and wants the option to route or fall back across backends.

Decision. Model access sits behind a `ModelProvider` interface; governance code never references a concrete backend. A mock provider returns deterministic, specialty-shaped output and is the default. A remote provider, selected by configuration, is the seam where a real backend is wired: an internal model gateway, a hosted model API, or a managed cloud service. Routing or fallback can be layered in front later.

Consequences. The backend can be swapped or composed without touching redaction, metering, policy, or audit. Tests run end to end with no external dependency and no model cost. Vendor choice is deferred to deployment configuration. Tradeoffs: one layer of indirection and a result contract that must stay stable across providers so token accounting stays uniform. A hardcoded provider was rejected.

ADR 0006 · ACCEPTED

Usage metering and soft budgets, not hard rate limits

Context. The plane needs visibility into cost and abuse: model calls carry real spend and a single tenant can drive large volume. The obvious approach, a hard rate limit, blocks a clinician or a patient-facing assistant mid-workflow and turns a cost concern into a reliability problem.

Decision. The default posture is metering, not throttling. Per-tenant request counts and token usage are recorded in Redis; crossing a soft budget is logged without blocking. A hard enforcement mode exists behind `app.usage.mode=enforce` for tenants or environments that explicitly opt into a cap. Enforcement is off by default.

Consequences. Assistants stay responsive by default and the cost signal feeds dashboards and alerts rather than outages. Operators can enable a hard ceiling per environment without code change. Tradeoff: metering alone does not stop a runaway tenant's in-the-moment spend, so it must pair with alerting and, where required, opt-in enforcement. The token counters are the foundation for per-tenant cost reporting and a future token budget.

SECTION 08

Acceptance criteria

- Every suggestion, on every path, has a complete audit trail with an actor for each transition, verified end to end by the Testcontainers integration test.
- No model call ever receives unredacted input; redaction is exercised by unit tests and cannot be disabled by configuration.
- Review policy changes require configuration only, never a code change, and are covered by tests on both the autonomous and review-required paths.
- The whole plane runs locally with no cloud dependency: Docker Compose for infrastructure, mock provider for the model, minikube for the cluster proof.

SECTION 09

Open items

- [ ] Replace the inline audit publish with the transactional outbox described in ADR 0003.
- [ ] Wire a production de-identification service behind `RedactionService` (candidates evaluated in CAG-05).
- [ ] Implement the remote provider against the chosen backend (options evaluated in CAG-05).
- [ ] Attach alerting to the soft-budget log signal so metering closes the loop operationally.
- [ ] Extend `GovernancePolicy` beyond assistant type: action risk, model confidence, tenant configuration.

END OF DOCUMENT · CAG-03